Q1 W.A.P to Create & Display a Singly Linked List :-

```
#include<iostream>
Using namespace Std;
Class node {
 Public;
 int data;
 node* next;

node( int data){
this -> data = data;
this -> next = nullptr;
}
};

Void insertHead( node*& head, int d){
   node* temp = new node(d);
   temp -> next = head;
   head = temp;
}

Void Print( node*& head){
   node* temp = head;
   While( temp != nullptr){
Cout << temp -> data << " ";
   temp = temp -> next;
}
```

```
int main() {
node* node1 = new node(10);
node* head = node1;
Print(head);
InsertHead(head, 20);
Print(head);
```

Sample Output :-

10

20 10

W.A.P of Linked List to add a Node at the Beginning.

```
#include <iostream>
Using namespace Std;
Class node {
  Public :
  int data;
  node * next;

 node (int data) {
   this -> data = data;
   this -> next = nullptr;
 }
};

Void insertTail ( node*& tail, int d) {
    node * temp = new node (d);
    tail -> next = temp;
    tail = temp;
}

Void Print ( node *& tail) {
    node * temp = tail;
  While ( temp != nullptr) {
    Cout << temp -> data << " ";
    temp = temp -> next;
}
```

```
int main() {
node* node1 = new node(10);
node* tail = node1;
Print(head);
insertTail(tail, 20);
Print(head);
insertTail(tail, 30);
Print(head);
}
```

Sample O/P :-

```
10
20 10
30 20 10
```

WAP to Insert a Node at the End of a S.L.L :-

```
#include<iostream>
Using namespace Std;
Class Node {
  Public:
  int data;
  node * next;

node(int data){
 this -> data = data;
this -> next = nullptr;

};

Void insertTail( node*& tail, int d){
   node * temp = new node(d);
    tail -> next = temp;
    tail = temp;

}

Void Print( node*& tail){
   node * temp = tail;
  While( temp != nullptr){
  Cout << temp -> data << " ";
  temp = temp -> next;

}
```

```
int main(){
node * node1 = new node(10);
node* tail = node1;
Print(head);
insertTail(tail, 20);
Print(head);
insertTail(tail, 30);
Print(head);
}
```

Sample O/P :-

```
10
10 20
10 20 30
```

W.A.P Insert a node at Given position.

```
#include<iostream>
Using namespace std;
 Class node{
  Public:
   int data;
   node* next;
  node(int data){
   this->data = data;
   this->next = nullptr;
  }
};
Void InsertHead(node*& head, int d){
   node* temp = new node(d);
   temp->next = head;
   head = temp;
}
  Void Print(node*& head){
     node* temp = head;
   while(temp!=nullptr){
     Cout<<temp->data<<" ";
     temp = temp->next;
   }
```

```
Void insertNode(node*& head, int pos
    node* temp = head;
    int cnt=1;
    while(cnt< position-1){
     temp = temp->next;
       cnt ++;
    }
    node* node2 = new node(d);
    node2->next = temp->next;
    temp->next = node2;
}
    int main(){
   node* nod1 = new node(10);
   node* head = node1;
   Print(head);
   InsertHead(head, 20);
   Print(head);
   InsertHead(head, 30);
   Print(head);
   insertNode(head, 2, 35);
   Print(head);
}
```

W.A.P Delete a node from the beginning.

```
#include<iostream>
Using namespace std;
    class node {
        Public:
        int data;
        node * next;
      node(int data){
      this-> data = data;
      this-> next= nullptr;
    }
    ~node(){
      int Value = this->data;
       if(this->next==nullptr){
            delete next;
          this->next = nullptr;
    }
}
};
Void insertHead(node*& head, int d){
      node * temp = new node(d);
       temp-> next = head;
       head = temp;
} Void insertTail(node*& tail, int d){
```

```
node * temp = new node(d);
tail -> next = temp;
tail = temp;
}

void Print( node*& head ) {
    node* temp = head;
    while( temp != nullptr) {
cout << temp -> data << " ";
    temp = temp -> next;
}
}

void insertNode ( node*& tail, node*& head, int posi,
                                            int d) {
    if ( posi == 1) {
        insertHead(head, d);
        return;
    }
    node * temp = head;
    int cnt = 1;
    while ( cnt < posi-1) {
    temp = temp -> next;
    cnt ++;
    }
    if ( temp -> next == nullptr) {
        InsertTail( tail, d);
        return;
    }
```

```
node * node2 = new node(d);
node2->next = temp->next;
temp->next = node2;
}

Void deleteNode(int posi, node* &head){
    if(posi==1){
        node* temp = head;
        head = head->next;
        temp->next = nullptr;
        delete head;
    }
    else{
        node* curr = head;
        node* pre = nullptr;
        int cnt=1;
        while(cnt<posi){
            pre = curr;
            curr = curr->next;
            cnt++;
        }
        pre->next = curr->next;
        curr->next = nullptr;
        delete curr;
    }
}
```

```
int main() {
node* node2 = new node(10);
 node* head = node2;
 Print(head);
 insertTail(tail, 20);
 Print(head);
 insertTail(tail, 30);
 Print(head);
insertNode(tail, head, 3, 50);
Print(head);
deleteNode(2, head);
Print(head);
```

Sample O/P :-

10
10 20
10 20 30
10 20 50 30
10 50 30

D.A.P Search for an Element in a Singly L.L ;-

```
#include<iostream>
using namespace std;
Struct Node {
    int data ;
    node * next ;
    node ( int val) : data (val) , next (nullptr)
    { }
};
int SearchElement ( node* head , int Key ){
    int pos = 1 ;
    Node* temp = head ;
while( temp != nullptr){
    if ( temp -> data == Key )
        return pos ;
    temp = temp -> next ;
    Pos++ ;
}
return -1 ;
}
Void display ( Node * head ){
    if ( head == nullptr ){
    cout <<"List is empty (NULL)\n";
    return ;
}
```

```
while (head != nullptr){
    cout << head -> data << " -> ";
    head = head -> next;
}
cout << "NULL\n";
}
void freeList (Node* head){
    while (head != nullptr){
        Node* temp = head;
        head = head -> next;
        delete temp;
    }
}
int main(){
    Node* head = nullptr;
    Node* tail = nullptr;
    int n, value, key;
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter " << n << " values: ";
    for(int i=0; i<n; i++){
        cin >> value;
    Node* newNode = new node(value);
```

```
if ( head == nullptr)  head = tail = newNode ;
  else {
        tail -> next = newNode ;
          tail = newNode ;

}
}
    cout<<"List : ";
    display (head);
    cout<<" Enter element to Search : ";
    cin >> Key ;
    int result = SearchElement (head, Key);
      if ( result != -1)
        cout<<" Element "<< Key <<" found at
      position "<< result <<" \n";
      else {
          cout<< " Element " << Key <<" not
                found in the List . \n ";
      freelist(head);
        return 0;
}
```

:- Sample O/P :-

Enter number : 5

Enter 5 number :- 10 20 30 40 50

Enter element to Search : 30

Element 30 found at position 3.

W.A.P Count Total Number of Nodes.

```
#include <iostream>
using namespace std;
Struct node {
    int data;
    Node * next;
    node ( int Val ) : data(Val), next(nullptr) {
};
int CountNode ( Node * head ) {
    Node* temp = head;
    While ( temp != nullptr ) {
        Count ++;
        temp = temp -> next;
    } return Count;
}
Void display ( Node* head ) {
    if ( head == nullptr ) {
        Cout << "List is empty (Null) \n";
        return;
    }
    While ( head != nullptr ) {
        Cout << head -> data << -> ";
        head = head -> next;
```

```
} cout << "Null \n";

} void freeList(Node* head){
    while (head != nullptr){
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

int main(){
    node* head = nullptr;
    node* tail = nullptr;
    int n, value;
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter " << n << " values: ";
    for (int i=0; i<n; i++){
        cin >> values;
    Node* newNode = new node(value)
    if (head == nullptr) head = tail =
            newNode;
    else {
        tail->next = newNode;
        tail = newNode;
    }
}
```

```
    }
}
    cout << " List : " ;
    display (head);
    cout << " Total number of nodes : " <<
            cout << Nodes (head) << " \n";
    freeList (head);
    return 0;
}
```

Sample O/P :-

Enter number of nodes : 5

Enter 5 values : 10 20 30 40 50

List 10 -> 20 -> 30 -> 40 -> 50 -> Null

Total Number of nodes : 5

Find Maximum & Minimum in L.L :-

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;
};
int main() {
    Node* head = nullptr;
    int n, val;
    cout << "Enter number of nodes : ";
    cin >> n;
    cout << "Enter " << n << " Values : ";
    while (n--) {
        cin >> val;
        head = new Node{val, head};
    }
    if (!head) { cout << "List is empty\n";
        return 0; }
    int mx = head->data, mn = head->data;
    for (node* t = head->next; t; t =
        t->next) {
```

```
    mx = max(mx, t->next);
    mn = min(mn, t->data);
  }
  cout << "Maximum : " << mx << "\n mi
       : << mn << "\n";
}
```

Sample O/P :-

```
Enter number of node : 5
Enter 5 Values : 80 10 50 20 40
Maximum : 50
Minimum : 10
```

Reverse a Singly L.L :-

```
#include <iostream>
Using namespace std;
Struct Node {
        int data;
        Node * next;
};
int main() {
    node * head = nullptr;
    int n, val;
cout << " Enter num of nodes : ";
 Cin >> n;
Cout << " Enter " << n << "Values : ";
node ** p = & head;
while( n-- ) {
    Cin >> val;
  *p = new Node(val, nullptr);
  p = &(*p) -> next;
}
node *prev = nullptr, *next;
while( head ) {
        next = head -> next;
        head -> next = prev;
        Prev = head;
```

```
        head = next;
    }
    head = Prev;

    cout<<" Reverse List : ";
    for (Node* t = head; t; t = t->next)
        cout << t->data << " ";
}
```

Sample O/P :-

5

10 20 30 40 50

50 40 30 20 10

Sort a L.L in ascending Order :-

```
#include<iostream>
Using namespace Std;

Struct Node {
    int data;
    node * next;

};

int main() {
    node * head = nullptr;
    int n, Val;
    Cout << " Enter num of nodes : ";
    Cin >> n;
    Cout << "  ";
    While (n--) {
        Cin >> Val;
        head = new node{Val, head};
    }

    for (node * i = head; i != i -> next)
        for ( node * j = head; j -> next; j = j -> next
            if ( j -> data > j -> next -> data)
                Swap ( j -> data, j -> next -> data);
    Cout << " Sorted list : ";
```

```
for ( node *t = head; t; t = t->next)
    cout << t->data << " ";
}
```

Sample O/P :-

```
Enter number of node : 5
Enter 5 Values 30 10 50 20 40
Sorted list : 10 20 30 40 50
```

Menu - Driven L.L :-

```
#include <iostream>
Using namespace std;
Struct node {
    int data;
    node * next;
};
    int main() {
        node * head = nullptr;
        int ch, val;
    do {
        Cout<< "In Insert 2, Delete 3 Search
            4, Display 5 Exit : ";
        Cin >> ch;
        if (ch == 1) {
            Cout<< "Value";
            Cin >> val;
            node ** p = &head;
        while (*p) p = &(*p) -> next;
        *p = new Node(val, nullptr);
    } else if (ch == 2) {
```

```
cout<<"Value to delete : "; cin>>val;
    node **p=&head;
    while (*p && (*p)->data != val)
      p=&(*p)->next;
    if (*p) {Node *t=*p; *p=t->next;
        delete t; cout<<"Delete In"; }

} else if (ch==3){
    cout<<"Value to Search : ";
      cin>>val;
    int pos=1;
    node* t=head;
    while(t && t->data != val)
      t=t->next,
      pos++;
  if(t) cout<<"found at position "<<pos<<
                "\n";
  else cout<<"not found \n";
} else if(ch==4){
  for(node* t=head;
      t=t->next) cout<<t->data<<"->";
    cout<<"Null";
```

Doubly L.L :-

```
#include <iostream>
Using namespace std;
Struct node {
    int data;
    node * pre, * next;
};
int main(){
    node * head = nullptr;
    int ch, val;
    do {
        cout << "\n 1. Insert 2. Delete 3. Display
                                4. Exit : ";
        cin >> ch;
        if ( ch == 1) {
            Cout << "Value : "; cin >> val;
            node * n = new node{ val, nullptr, head };
            if ( (head)  head -> prev = n;
            head = n;
        } else if( ch == 2) {
            Cout << " Value to delete : ";
            cin >> val;
            node * t = head;
```

```
While (t && t->data != val)
    t = t->next;
if(!t) { cout << "Not found\n"; continue; }
if(t->prev) t->prev->next = t->next;
else head = t->next;
if(t->next) t->next->prev = t->prev;
    delete t;
  cout << "Deleted\n";
}
else if (ch == 3) {
      cout << "Null <-> ";
  for( node* t = head;
    t = t->next )
    cout << t->data << " <-> ";
    cout << "null\n";
}
while(ch != 4);
}
```

Circular L.L :-

```
#include <iostream>
Using namespace std;
Struct node {
    int data;
    node * next;
};
int main() {
node* last= nullptr;
 int ch, val;
do {
    cout << "\n Insert at end 2 traverse 3 Exit :"
    cin >> ch;
    if ( ch==1)
      cout << " Value";
    cin >> val;
    node* n = new Node {val, nullptr};
    if (! last) n->next = n;
    else { n->next = last->next;
      last->next =n; }
      last = n;
    }
    else if(ch==2)
    if (!last) { cout << " List is
          empty \n"; continue;}
```

```
node * t = last -> next;
do { cout << t -> data << " -> ";
  t = t -> next; }
while ( t != last -> next);
 cout << " (back to head) \n";
 }
 } while (ch != 3);
 }
```

Sample O/P :-

1 Insert at end 2. Traverse 3 Exit 1

Value 10

1 Insert at end 2. Traverse 3 Exit 1

Value : 20

1 Insert at end 2. Traverse 3. Exit : 1

Value : 30

1 Insert at end 2 Traverse 3 Exit : 2

10 -> 20 -> 30 -> (back to head)